

ECE 51216 Project: DPLL SAT Solver with Heuristics

Group 19: Samitha Ranasinghe, Milen Manoj

I. INTRODUCTION

Boolean Satisfiability (SAT) is a fundamental problem in computer science and electronic design automation, where the objective is to determine whether a given Boolean formula can be satisfied by some assignment of truth values to its variables. Despite its simple definition, SAT is known to be NP-complete and serves as the foundation for solving many real-world problems such as hardware verification, scheduling, and optimization.

In this project, we focus on implementing a SAT solver based on the Davis–Putnam–Logemann–Loveland (DPLL) algorithm. The goal is to take a Boolean formula in Conjunctive Normal Form (CNF), provided in DIMACS format, and determine whether it is satisfiable. If the formula is satisfiable, the solver should return a valid assignment of variables; otherwise, it should correctly identify the formula as unsatisfiable.

Our approach begins with constructing a reliable input parsing system to interpret DIMACS CNF files. We then build a decision tree representing possible variable assignments and apply a basic version of the DPLL algorithm to explore this search space. This structured approach allows us to better understand how SAT solvers systematically evaluate assignments and detect contradictions.

After implementing the basic DPLL solver, we extended the solver with advanced SAT solving techniques. These include conflict-driven clause learning, non-chronological backtracking, and two-watched literals for efficient Boolean Constraint Propagation. These additions allow the solver to learn from conflicts, avoid repeated mistakes, and reduce unnecessary clause scanning during propagation.

II. METHODS

The implementation of our SAT solver is divided into several main components: input parsing, decision tree construction, the basic DPLL algorithm, conflict-driven clause learning, non-chronological backtracking, and efficient Boolean Constraint Propagation using watched literals.

A. DIMACS Input Parsing

The first step in our solver is to read and interpret CNF formulas provided in DIMACS format. The parser processes the input file by ignoring comment lines and extracting key information from the problem definition line, including the number of variables and clauses. Each clause is then read as a list of integers, where positive integers represent variables and negative integers represent their negations.

One of the main challenges in this step was handling variations in DIMACS formatting. Some files contain clauses spread across multiple lines or include extra whitespace and comments. To address this, we designed the parser to process input line-by-line and correctly group literals until a terminating zero is encountered. This ensured robustness across different benchmark files.

The output of the parser is a structured representation of the formula as a list of clauses, which is then used throughout the solver.

B. Decision Tree Construction

After parsing the input, we construct a decision tree that represents the space of possible assignments of variables. Each level of the tree corresponds to a variable, and each node represents a partial assignment. The left and right branches correspond to assigning a variable as true or false, respectively.

The tree is built recursively, ensuring that all combinations of assignments are represented for smaller test cases. Each node stores the current assignment of variables, allowing the solver to track decisions made along each path.

A key challenge in this stage was managing the exponential growth of the tree. Since the number of nodes grows as 2^n , constructing a full tree quickly becomes infeasible for larger inputs. To handle this, we limited our tree construction to smaller test cases and focused on ensuring correctness and clarity of structure rather than scalability at this stage.

Figure 1 illustrates a simplified example of the decision tree for three variables.

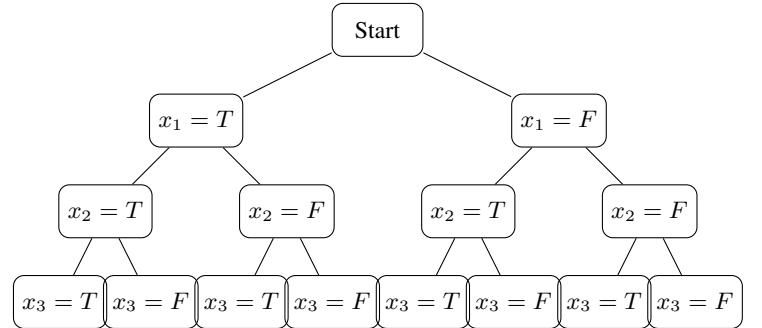


Fig. 1: Decision tree for three Boolean variables

C. Basic DPLL Algorithm

The DPLL algorithm is used to traverse the decision tree and evaluate the satisfiability of the CNF formula. At each node,

the current assignment is applied to simplify the set of clauses. Clauses that evaluate to true are removed, while literals that evaluate to false are eliminated from their respective clauses.

The algorithm checks for two key conditions: if all clauses are satisfied, the formula is declared satisfiable; if any clause becomes empty, the current assignment leads to a contradiction and is therefore unsatisfiable. If neither condition is met, the algorithm continues exploring the tree by branching on the remaining variables.

One challenge in implementing this step was ensuring that clause simplification was performed correctly at each stage without modifying the original formula. This required careful handling of copies of clause lists and assignments to avoid unintended side effects during recursion.

Overall, this basic implementation of DPLL provides a clear understanding of how SAT solvers operate, even though it does not yet include advanced heuristics for optimization.

D. Basic DPLL Pseudocode

The basic DPLL procedure can be summarized as follows:

```
function DPLL(clauses, assignment):
    apply Boolean Constraint Propagation

    if any clause is empty:
        return UNSAT

    if all clauses are satisfied:
        return SAT

    choose an unassigned variable x

    assign x = True
    if DPLL(clauses, assignment):
        return SAT

    assign x = False
    if DPLL(clauses, assignment):
        return SAT

    return UNSAT
```

This basic recursive version provided the foundation for our solver before advanced heuristics were added.

E. Conflict-Driven Clause Learning

To improve the efficiency of the solver, we implemented conflict-driven clause learning. In a basic DPLL solver, when a conflict is found, the solver simply backtracks and tries another branch. However, this can cause the solver to repeat similar mistakes later in the search.

Conflict-driven clause learning addresses this issue by analyzing the cause of a conflict and creating a new learned clause. This learned clause is added back into the clause database so that the solver avoids repeating the same conflicting assignment pattern.

In our implementation, conflict analysis uses a First-UIP style approach. The solver starts with the conflicting clause and

traverses the implication trail backward. It repeatedly resolves the conflict clause with antecedent clauses until only one literal from the current decision level remains. The resulting learned clause is called an asserting clause because it forces a new implication after the solver backtracks.

F. Conflict Analysis Pseudocode

```
function FIX_CONFLICT(conflict_clause):
    learned_clause = conflict_clause

    while learned_clause has more than one
        literal from the current decision level:

        choose the most recent assigned literal
        from the current decision level

        antecedent_clause = reason for that
        assignment

        learned_clause = resolve(learned_clause,
        antecedent_clause)

    backjump_level = highest decision level in
    learned_clause excluding current level

    return learned_clause, backjump_level
```

This process improves the quality of learned clauses and helps the solver avoid redundant branches.

G. Non-Chronological Backtracking

In basic DPLL, backtracking is chronological. This means that when a conflict occurs, the solver returns only to the immediately previous decision level. However, the most recent decision is not always the true cause of the conflict.

To address this, we implemented non-chronological backtracking, also called backjumping. After conflict analysis, the solver determines the decision level most relevant to the learned clause and jumps directly to that level. This allows the solver to skip decision levels that are not responsible for the conflict.

This technique works together with conflict-driven clause learning. The learned clause explains the cause of the conflict, and the computed backjump level tells the solver where to resume the search.

H. Non-Chronological Backtracking Pseudocode

```
function BACKTRACK(level):
    while trail is not empty:
        last_literal = most recent assignment

    if decision_level(last_literal) <= level:
        stop

    remove last_literal from assignment
    remove last_literal from trail
```

```
current_decision_level = level
```

This allows the solver to avoid unnecessary chronological backtracking and improves search efficiency.

I. Boolean Constraint Propagation Using Watched Literals

Boolean Constraint Propagation is one of the most frequently used operations in a SAT solver. In a naive implementation, the solver scans all clauses after every assignment to check whether any clause has become unit or conflicting. This becomes expensive for large CNF formulas.

To improve propagation efficiency, we implemented the two-watched literals technique. Each clause watches two literals. When a literal becomes false, only the clauses watching that literal need to be examined. If another non-false literal can be found in the clause, the watch is moved. If no replacement exists, the clause is either unit or conflicting.

This optimization significantly reduces the amount of unnecessary clause scanning. Learned clauses are also inserted into the watched literal data structure so that they participate in future propagation.

J. Watched Literals Pseudocode

```
function UNIT_PROPAGATE_WATCHED():
    while propagation queue is not empty:
        lit = pop from propagation queue
        false_lit = negation of lit

        for each clause watching false_lit:
            other_watch = other watched literal

            if other_watch is True:
                continue

            if another non-false literal exists:
                move watch to that literal

            else if other_watch is unassigned:
                assign other_watch

        else:
            return conflict clause

    return no conflict
```

K. Data Structures Used

The main data structures used in the solver are:

- **Clause list:** Stores the original CNF clauses parsed from the DIMACS file.
- **Learned clause list:** Stores clauses generated during conflict analysis.
- **Assignment map:** Stores the current Boolean assignment of each variable.
- **Decision level map:** Records the decision level at which each variable was assigned.
- **Antecedent map:** Stores the clause responsible for each implied assignment.

- **Trail:** Maintains the chronological sequence of assigned literals.
- **Watched literal map:** Maps each watched literal to the clauses currently watching it.
- **Clause watch map:** Stores the two watched literals for each clause.
- **Propagation queue:** Stores assigned literals whose implications still need to be processed.

These data structures were chosen because they directly support efficient propagation, conflict analysis, clause learning, and backtracking.

III. EXPERIMENTAL METHODOLOGY

The solver was tested against a variety of instances from the SATLIB library to evaluate its performance on both random and structured problems:

- **Random 3-SAT:** uf20, uf50, uf100
- **Structured Graphs:** flat125-301
- **Pigeonhole Problems:** hole6, hole7

To isolate the performance impact of each heuristic, we performed an ablation study by executing the solver in four configurations: **Base DPLL**, **+ Learning** (CDCL), **+ Backtrack** (Backjumping), and **Full CDCL** (incorporating Two-Watched Literals). Four primary metrics were recorded: execution time, decision count, total implications, and the speedup ratio relative to the baseline. Each testcase was also run on MiniSat in order to compare best performance.

- Basic DPLL without advanced heuristics.
- DPLL with conflict-driven clause learning.
- CDCL with First-UIP conflict analysis and non-chronological backtracking.
- CDCL with watched literals and propagation queue.
- MiniSat

All tests were run under the same general setup so that runtime comparisons would be meaningful. The main metric used was execution time in seconds.

IV. RESULTS

The pipeline was utilized to verify SATISFIABLE models by plugging literals back into the clauses to ensure validity. For UNSATISFIABLE instances, results were cross-validated against MiniSat.

As shown in Table I, the transition from a baseline DPLL implementation to a full Conflict-Driven Clause Learning (CDCL) solver with Two-Watched Literals resulted in massive performance gains. For the uf100-430 dataset, the baseline averaged 995.5 decisions, which was reduced to 241.3 decisions in the optimized version, yielding a 96.51x speedup. The most dramatic improvement was observed in the hole6 benchmark, where the speedup reached 376.93x. In the case of hole7, the baseline DPLL failed to converge within the 120-second timeout, whereas the full CDCL implementation solved the instance in under 3 seconds.

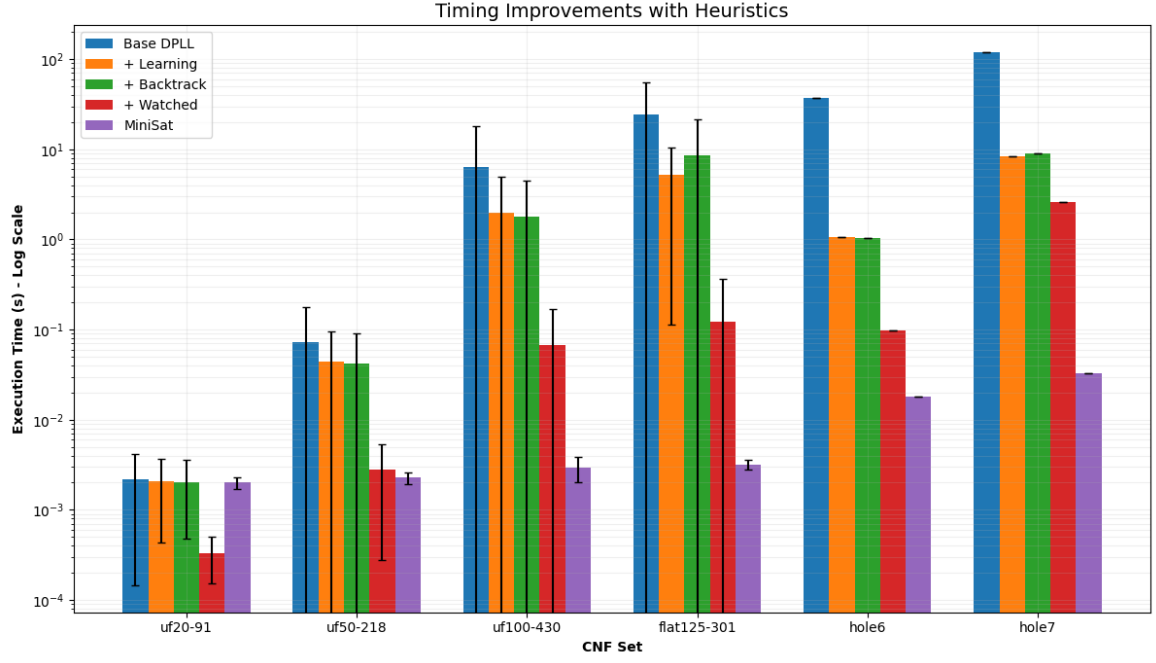


Fig. 2: Average execution time across benchmarks (Logarithmic Scale)

Benchmark	Config	Avg Decisions	Avg Implications	Speedup
uf20-91	Base DPLL	8.8	35.2	-
	+ Learning	8.4	32.1	1.06x
	+ Backtrack	8.6	31.3	1.07x
	+ Watched	8.7	30.7	6.61x
uf50-218	Base DPLL	43.5	547.3	-
	+ Learning	27.2	298.0	1.64x
	+ Backtrack	29.2	276.9	1.75x
	+ Watched	31.0	276.5	25.97x
uf100-430	Base DPLL	995.5	26007.3	-
	+ Learning	179.5	4791.6	3.29x
	+ Backtrack	197.5	4480.2	3.63x
	+ Watched	241.3	4622.6	96.51x
flat125-301	Base DPLL	329.3	27301.9	-
	+ Learning	96.9	6871.1	4.60x
	+ Backtrack	177.2	10403.2	2.80x
	+ Watched	249.2	12222.9	195.68x
hole6	Base DPLL	50589.0	679191.0	-
	+ Learning	299.0	4968.0	35.17x
	+ Backtrack	362.0	5142.0	35.64x
	+ Watched	659.0	5795.0	376.93x
hole7	Base DPLL	-	-	(timeout of 120s)
	+ Learning	793.0	15909.0	14.51x
	+ Backtrack	938.0	17641.0	13.23x
	+ Watched	4181.0	29764.0	46.52x

TABLE I: Performance metrics and speedup ratios across benchmark sets.

TABLE II: BCP Propagation Throughput

Benchmark Set	Avg. Implications/sec
uf20-91	93,350.56
uf50-218	99,054.37
uf100-430	69,395.80
flat125-301	98,937.04
hole6	58,653.85
hole7	11,539.12

Driven Clause Learning (1-UIP) slashed decision counts by over 99% in structured instances like *hole6*. However, the implementation of Two-Watched Literals provided the most significant speedup by removing the BCP bottleneck. Our analysis shows a 2WL throughput of nearly 100,000 implications per second, allowing the solver to explore the search space an order of magnitude faster than a linear clause scan.

V. ANALYSIS OF RESULTS

The experimental data reveals a clear distinction between logic-based pruning and data-driven throughput. Conflict-

VI. CONCLUSIONS

This project successfully implemented a high-performance CDCL SAT solver. The results confirm that while conflict

learning is essential for pruning the search tree, the mechanical efficiency of the BCP engine, specifically through the use of Two-Watched Literals, is the primary determinant of real-world solver speed.

VII. FUTURE WORK

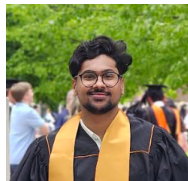
Although the current solver implements DPLL, conflict-driven clause learning, non-chronological backtracking, and watched literals, there are still several possible improvements. One future improvement is to add a stronger branching heuristic such as VSIDS, which would prioritize variables involved in recent conflicts. Another improvement is clause database management, where older or less useful learned clauses are periodically removed to reduce memory growth.

ACKNOWLEDGMENT

In this project, we used ChatGPT/Google Gemini as a supporting tool to help improve the grammar, formatting, and overall structure of our report. We also used it to assist with implementing supporting components such as the DIMACS input parser and benchmarking scripts. All core algorithm design, including the implementation of the SAT solver, conflict-driven clause learning, non-chronological backtracking, and watched literals, was completed independently by us.



Samitha Ranasinghe is pursuing his M.S. in Electrical and Computer Engineering at Purdue University. His research interests include computer architecture and robotics.
PUID: 0032936721



Milen Manoj is pursuing his M.S. in Computer Engineering at Purdue University. His research interests include computer systems and computer architecture.
PUID: 0033578671